SEMANA 4

ACTIVIDAD 1 *Visualización de Lista Doblemente Enlazada*

- Documento con los 3 diagramas anotados por ti (puedes hacerlos en papel, ASCII o herramienta digital). Incluye una explicación con tus propias palabras de por qué eliminar un nodo conocido es O(1) en la lista doble pero O(n) en la lista simple.

Operación 1: Insertar 3 al inicio

Antes

null ◀– [7] ◀—▶ [15] ◀—▶ [23] ◀—▶ [42] –▶ null

Cambios de punteros

- [3].next → [7]
- [3].prev → null
- [7].prev → [3]
- cabeza → [3]

Después

null ◀– [3] ◀—▶ [7] ◀—▶ [15] ◀—▶ [23] ◀—▶ [42] –▶ null cabeza = [3], cola = [42]

Pseudocódigo

newNode = Node(3) newNode.next = head newNode.prev = null head.prev = newNode head = newNode

Complejidad: O(1)
Cabeza cambió: Sí
Cola cambió: No

Operación 2: Insertar 50 al final

Antes

null ◀– [3] ◀—▶ [7] ◀—▶ [15] ◀—▶ [23] ◀—▶ [42] –▶ null

Cambios de punteros

- [50].prev → [42]
- [50].next → null
- [42].next → [50]
- cola → [50]

Después

null ◀– [3] ◀—▶ [7] ◀—▶ [15] ◀—▶ [23] ◀—▶ [42] ◀—▶ [50] –▶ null cabeza = [3], cola = [50]

Pseudocódigo

newNode = Node(50) newNode.prev = tail newNode.next = null tail.next = newNode tail = newNode

Complejidad: O(1)
Cabeza cambió: No
Cola cambió: Sí

Operación 3: Eliminar nodo [15]

Antes

null ◀– [3] ◀—▶ [7] ◀—▶ [15] ◀—▶ [23] ◀—▶ [42] ◀—▶ [50] –▶ null

Cambios de punteros

- [7].next → [23]
- [23].prev → [7]
- [15] queda fuera

Después

null ◀– [3] ◀—▶ [7] ◀—▶ [23] ◀—▶ [42] ◀—▶ [50] –▶ null cabeza = [3], cola = [50]

Pseudocódigo

function deleteNode(node): node.prev.next = node.next node.next.prev = node.prev free(node)

Complejidad: O(1)
Cabeza cambió: No
Cola cambió: No

Recorrido final

De cabeza a cola:

[3] → [7] → [23] → [42] → [50]

De cola a cabeza:

[50] → [42] → [23] → [7] → [3]

Imagina que cada nodo es un vagón de tren con enganches adelante y atrás.

- En una lista doblemente enlazada, si ya estás parado en el vagón que quieres quitar, basta con soltar sus enganches y unir directamente a sus vecinos. Eso es O(1) porque no importa cuántos vagones haya: solo ajustas dos conexiones.
- En una lista simplemente enlazada, los vagones solo tienen enganche hacia adelante. Para quitar uno, necesitas saber quién lo engancha por detrás. Como no hay puntero prev, tienes que recorrer desde el inicio hasta encontrar al vagón anterior. Eso cuesta tiempo proporcional al número de nodos, es decir O(n).

ACTIVIDAD 2 *Proyecto: Playlist Bidireccional con Lista Doble propia*

- Entregable único: Código fuente del sistema Playlist Bidireccional con la lista doble construida por ti. Incluye: (1) la clase NodoDoble y la estructura ListaDoble con mínimo 10 métodos funcionales, (2) la clase Playlist con los 5 métodos del escenario, (3) una demostración que ejecute una secuencia de operaciones mostrando inserción, navegación adelante/atrás, eliminación de canción actual y recorrido completo, (4) comentarios explicando la lógica de cada bloque y cómo el puntero "actual" se mantiene consistente.

```
#include <iostream>

#include <string>

using namespace std;

class NodoDoble {

public:

   string valor;

   NodoDoble* next;

   NodoDoble* prev;

   NodoDoble(string v) {

      valor = v;

      next = nullptr;

      prev = nullptr;

   }
```

```
};
class ListaDoble {
private:
    NodoDoble* head;
    NodoDoble* tail;
public:
    ListaDoble() {
        head = nullptr;
        tail = nullptr;
    }
    void insertarInicio(string v) {
        NodoDoble* nuevo = new NodoDoble(v);
        nuevo->next = head;
        if (head != nullptr) head->prev = nuevo;
        else tail = nuevo;
        head = nuevo;
    }
    void insertarFinal(string v) {
        NodoDoble* nuevo = new NodoDoble(v);
        nuevo->prev = tail;
        if (tail != nullptr) tail->next = nuevo;
        else head = nuevo;
        tail = nuevo;
    }
```

```
void eliminarInicio() {
    if (head == nullptr) return;
    NodoDoble* temp = head;
    head = head->next;
    if (head != nullptr) head->prev = nullptr;
    else tail = nullptr;
    delete temp;
}
void eliminarFinal() {
    if (tail == nullptr) return;
    NodoDoble* temp = tail;
    tail = tail->prev;
    if (tail != nullptr) tail->next = nullptr;
    else head = nullptr;
    delete temp;
}
void eliminarNodo(NodoDoble* nodo) {
    if (nodo == nullptr) return;
    if (nodo->prev != nullptr) nodo->prev->next = nodo->next;
    else head = nodo->next;
    if (nodo->next != nullptr) nodo->next->prev = nodo->prev;
    else tail = nodo->prev;
    delete nodo;
```

```
}

NodoDoble* buscar(string v) {
    NodoDoble* actual = head;
    while (actual != nullptr) {
        if (actual->valor == v) return actual;
        actual = actual->next;
    }
    return nullptr;
}
void recorrerAdelante() {
    NodoDoble* actual = head;
    while (actual != nullptr) {
        cout << actual->valor << " ";
        actual = actual->next;
    }
    cout << endl;
}
void recorrerAtras() {
    NodoDoble* actual = tail;
    while (actual != nullptr) {
        cout << actual->valor << " ";
        actual = actual->prev;
    }
```

```
        cout << endl;
    }
    NodoDoble* getHead() { return head; }
    NodoDoble* getTail() { return tail; }
};
class Playlist {
private:
    ListaDoble lista;
    NodoDoble* actual;
public:
    Playlist() {
        actual = nullptr;
    }
    void agregarCancion(string nombre) {
        lista.insertarFinal(nombre);
        if (actual == nullptr) actual = lista.getHead();
    }
    void siguiente() {
        if (actual == nullptr) {
            cout << "Playlist vacía" << endl;
            return;
        }
        if (actual->next == nullptr) {
            cout << "Fin de playlist" << endl;
```

```
    } else {
        actual = actual->next;
        cout << "Reproduciendo: " << actual->valor << endl;
    }
}
void anterior() {
    if (actual == nullptr) {
        cout << "Playlist vacía" << endl;
        return;
    }
    if (actual->prev == nullptr) {
        cout << "Inicio de playlist" << endl;
    } else {
        actual = actual->prev;
        cout << "Reproduciendo: " << actual->valor << endl;
    }
}
void eliminarActual() {
    if (actual == nullptr) {
        cout << "Playlist vacía" << endl;
        return;
    }
    NodoDoble* temp = actual;
    if (actual->next != nullptr) {
```

```
            actual = actual->next;
        } else {
            actual = actual->prev; // si no hay siguiente, retroceder
        }
        lista.eliminarNodo(temp);
        if (actual != nullptr)
            cout << "Reproduciendo: " << actual->valor << endl;
        else
            cout << "Playlist vacía tras eliminación" << endl;
    }
    void mostrarPlaylist() {
        NodoDoble* cursor = lista.getHead();
        while (cursor != nullptr) {
            if (cursor == actual) cout << "*";
            cout << cursor->valor << " ";
            cursor = cursor->next;
        }
        cout << endl;
    }
};
int main() {
    Playlist p;
    p.agregarCancion("Song A");
    p.agregarCancion("Song B");
```

```
    p.agregarCancion("Song C");

    p.agregarCancion("Song D");

    cout << "Playlist inicial: ";

    p.mostrarPlaylist();

    p.siguiente();

    p.siguiente();

    p.anterior();

    p.eliminarActual();

    cout << "Playlist final: ";

    p.mostrarPlaylist();

    return 0;

}
```

- ListaDoble implementa los 10 métodos esenciales (insertar, eliminar, buscar, recorrer, obtener cabeza/cola).

- Playlist reutiliza esa lista y mantiene un puntero que siempre apunta a la canción en reproducción.

- Al eliminar la canción actual, el puntero salta a la siguiente si existe; si no, retrocede a la anterior.

- Los casos frontera (lista vacía, inicio, fin) se manejan con mensajes claros.

ACTIVIDAD 3 *El Auditor de Listas Dobles*

- *Documento con las 3 implementaciones erróneas, tu análisis de cada error (dónde está, por qué es incorrecto, qué consecuencia tendría en ejecución), y la versión corregida de cada método.*

***Insertar()final* con bug sutil en punteros**

Código erróneo

```
void insertarFinal(int valor) {
```

```
    Nodo* nuevo = new Nodo{valor, nullptr, nullptr};

    if (!cabeza) {

        cabeza = cola = nuevo;

    } else {

        cola->siguiente = nuevo;

        nuevo->anterior = cola;

        cola = nuevo;    }

}
```

Análisis del error

- Dónde está: El método no actualiza el puntero nuevo->siguiente en el caso general.
- Por qué es incorrecto: Aunque nuevo->siguiente se inicializa como nullptr, si la lista tenía más elementos y luego se hacen operaciones de recorrido hacia adelante y atrás, puede haber inconsistencias si se reutiliza el nodo o si se esperaba que la cola anterior apuntara correctamente.
- Consecuencia: La lista puede quedar con referencias inconsistentes al recorrer desde la cola hacia adelante, especialmente si se intenta insertar más nodos después.

Versión corregida

```
void insertarFinal(int valor) {

    Nodo* nuevo = new Nodo{valor, nullptr, nullptr};

    if (!cabeza) {

        cabeza = cola = nuevo;

    } else {

        cola->siguiente = nuevo;

        nuevo->anterior = cola;

        nuevo->siguiente = nullptr;

        cola = nuevo;

    }
```

```
}
```

*Eliminar()nodo*que falla en un caso frontera

Código erróneo

```
void eliminarNodo(int valor) {
    Nodo* actual = cabeza;
    while (actual) {
        if (actual->dato == valor) {
            if (actual->anterior) {
                actual->anterior->siguiente = actual->siguiente;
            }
            if (actual->siguiente) {
                actual->siguiente->anterior = actual->anterior;
            }
            delete actual;
            return;
        }
        actual = actual->siguiente;
    }
}
```

Análisis del error

- Dónde está: No se actualiza cabeza ni cola cuando el nodo eliminado es el primero o el último.
- Por qué es incorrecto: Si eliminas la cabeza, cabeza sigue apuntando a un nodo borrado. Si eliminas la cola, cola también queda inválida.
- Consecuencia: La lista queda corrupta en operaciones posteriores (recorridos, inserciones).

Versión corregida

```
void eliminarNodo(int valor) {
    Nodo* actual = cabeza;
    while (actual) {
        if (actual->dato == valor) {
            if (actual->anterior) {
                actual->anterior->siguiente = actual->siguiente;
            } else {
                cabeza = actual->siguiente;
            }
            if (actual->siguiente) {
                actual->siguiente->anterior = actual->anterior;
            } else {
                cola = actual->anterior;
            }
            delete actual;
            return;
        }
        actual = actual->siguiente;
    }
}
```

***Insertar()después* que rompe la simetría bidireccional**

Código erróneo

```
void insertarDespues(Nodo* nodo, int valor) {
```

```
    if (!nodo) return;

    Nodo* nuevo = new Nodo{valor, nullptr, nullptr};

    nuevo->siguiente = nodo->siguiente;

    nodo->siguiente = nuevo;

    nuevo->anterior = nodo;

    // BUG: la simetría se rompe aquí, parece correcto pero no lo es

}
```

Análisis del error

- Dónde está: No se actualiza el puntero anterior del nodo que originalmente estaba después de nodo.
- Por qué es incorrecto: La relación bidireccional se rompe: nuevo->siguiente->anterior no apunta a nuevo.
- Consecuencia: Al recorrer hacia atrás desde el nodo siguiente, se pierde la referencia correcta y la lista queda inconsistente.

Versión corregida

```
void insertarDespues(Nodo* nodo, int valor) {

    if (!nodo) return;

    Nodo* nuevo = new Nodo{valor, nullptr, nullptr};

    nuevo->siguiente = nodo->siguiente;

    nuevo->anterior = nodo;

    nodo->siguiente = nuevo;

    if (nuevo->siguiente) {

        nuevo->siguiente->anterior = nuevo;

    }

}
```

Conclusión

- insertarFinal(): Bug sutil por no asegurar puntero siguiente explícito.
- eliminarNodo(): Bug en casos frontera (cabeza y cola).
- insertarDespues(): Bug de simetría bidireccional.

Cada error refleja un “enganche mal puesto” en el tren de vagones. La corrección asegura que los vagones siempre estén bien conectados hacia adelante y hacia atrás.